Project Report: Group 014-3

Application Name

Web-E-Vil

Group Members

Samuel Lauer, Sami Yohannes, Macallan Finigan, Denys Davydenko

Summary

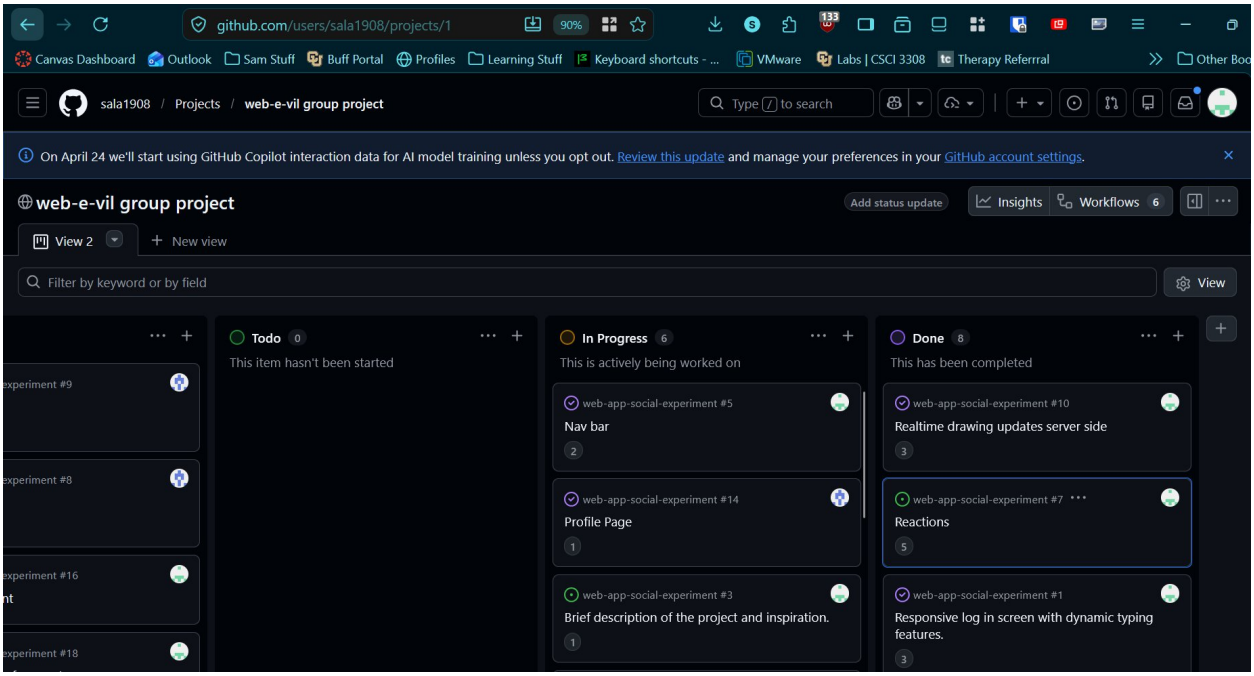
Web-E-Vil was inspired by the social experiment, r/place, in which a community on the website Reddit, was dedicated to a large canvas where users were limited to one pixel change per 5 minutes, which prompted large scale community involvement to make flags, memes, and many other fun evolving pieces of art.

In Web-E-Vil, users are given a limited number of pixels a day to draw on a 1024x1024 canvas, which updates in real time for all other users. As users play more, they are rewarded with level ups. When a user levels up, they are given a token which they can use to unlock new color palettes from the palette store. Additionally, as they level up they're given more pixels to draw per day.

Users are given a protective bubble around any piece of art larger than 3 pixels, which stops players from over writing each others work, and allows interaction by just clicking on another users artwork, you can give them a like or dislike, and report the user. This also allows admin accounts to remove a specific piece of art, or ban a user which will then remove all of their work from the canvas.

Project Tracker

https://github.com/users/sala1908/projects/1/views/2



Video Demonstration (link to youtube?)

VCS

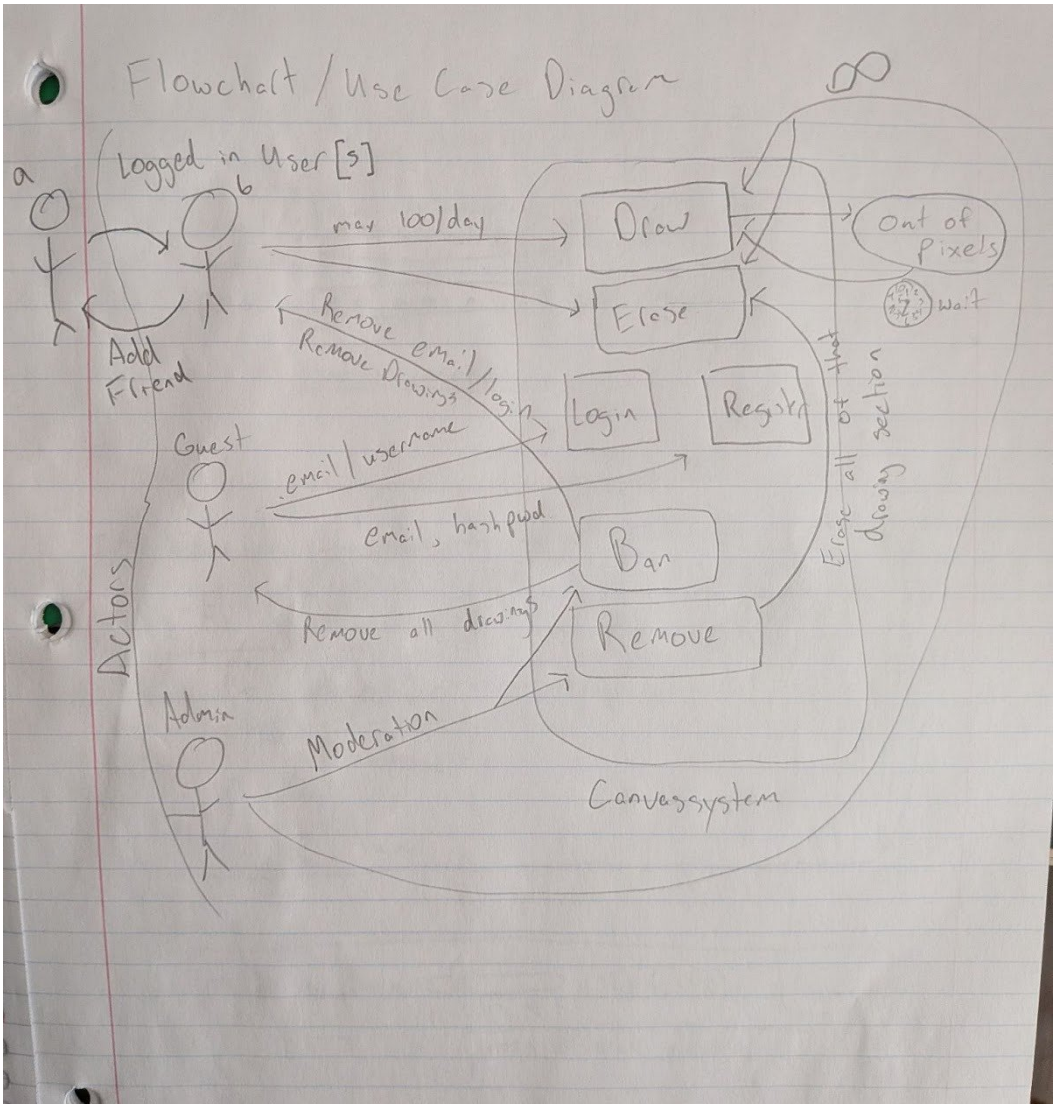
https://github.com/sala1908/web-app-social-experiment

Contributions

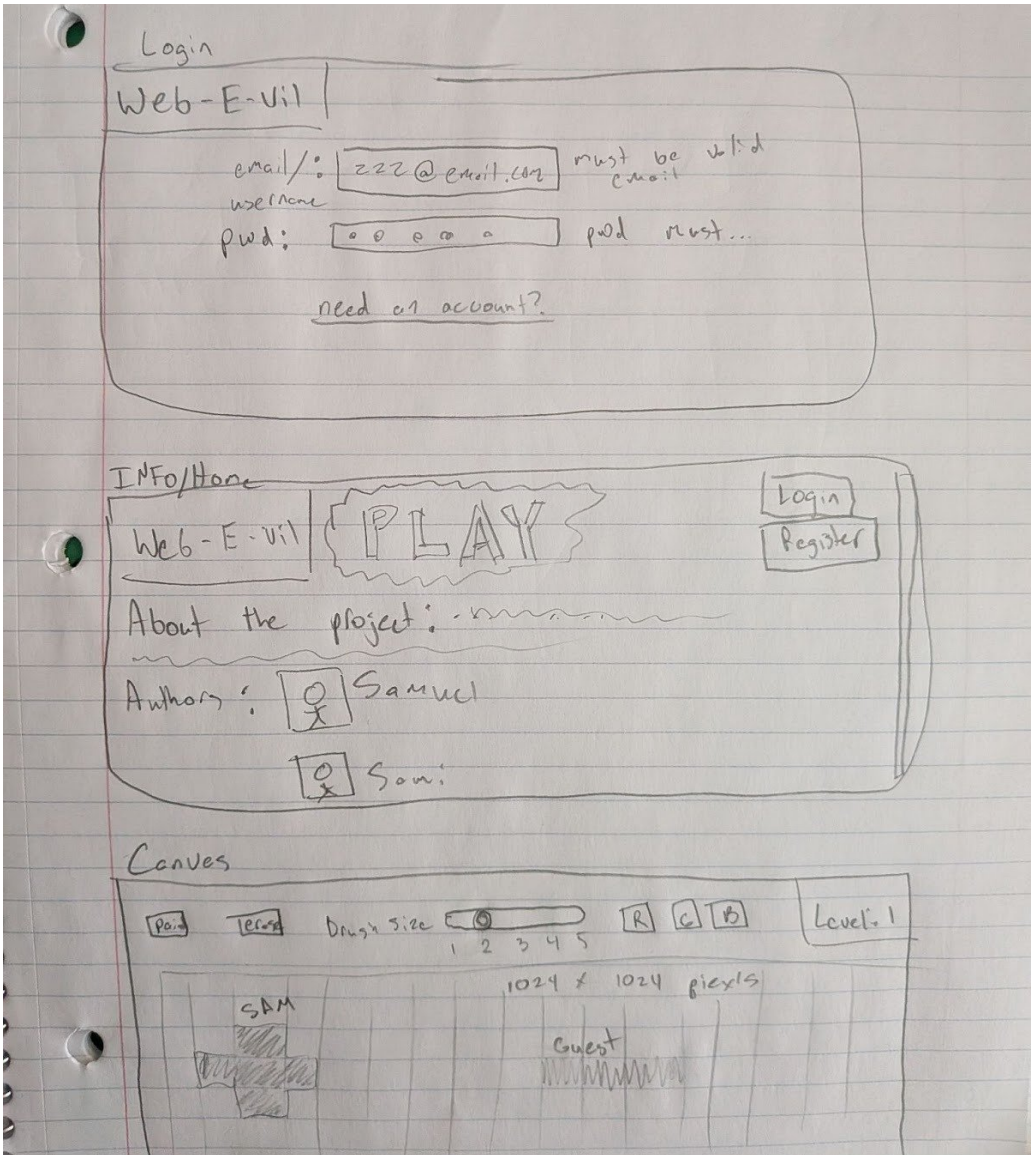
Samuel Lauer:

Technologies worked on- Docker/React/Express/NodeJS/Sockets. Using docker containers I made the web app and data base system, then using Sockets and NodeJS I connected them all together to make the Web-E-Vil application. I created the testing suite during Lab 10, and then iterated on that as development continued.

Use Case Diagram



Wireframes



UAT Results

UAT was executed for all scenarios documented above in the defined environment. All expected outcomes were observed.

Test 1 - Color Palette Rules: Verified palette enforcement for paint actions (individual-user palette or level-based palette, depending on active implementation). Execution used GET /api/palette to confirm allowed colors, followed by repeated POST /api/paint

calls for allowed and disallowed colors, with DB checks on canvas_pixels, paint_actions, and pixel_history after each step.

Test 2 - Daily Paint Limit Enforcement: Verified that regular users can paint only up to the configured daily cap and then receive 429. Execution used GET /api/me/limits before and after repeated valid POST /api/paint requests, then one extra over-limit request, with DB validation that rejected attempts did not add new paint_actions or pixel_history rows.

Test 3 - Admin Controls and Safety: Verified admin-only access to reset endpoints and correct post-reset behavior. Execution first attempted POST /api/admin/reset-canvas and POST /api/admin/reset-daily-limit as a regular user (403 expected), then repeated as admin (200 expected), validated DB resets, and confirmed normal user painting still succeeded afterward.

Test 4 - Authentication and Form Validation: Verified successful registration/login with valid credentials, rejection of invalid credentials, and specific validation feedback for missing required fields. Execution used POST /api/auth/register and POST /api/auth/login with positive and negative cases, plus DB checks to confirm correct writes to users only on valid registration.

No blocking defects or unexpected behavior were identified, and all listed acceptance criteria were met.

Deployment

https://web-app-social-experiment.onrender.com/